Nnomm Flash-Deal Claim Service

Unified Master Architecture, System Design & Staff-Level Technical Interview Guide

Build Status: 22/22 Automated Concurrency & Web Tests Passing (100%) | Stack: Spring Boot 3, Kotlin, PostgreSQL 15, HikariCP, Flyway, Testcontainers

> **Production Readiness & Verification Summary:**
> This unified master document contains the complete system architecture, real-world failure proofs, exact code line references, technology decisions, modification cheatsheet, and the 7 candidate brief answers for the Nnomm engineering evaluation.

Table of Contents

1. **0. The Real-World Business Problem & Core Challenge**
2. **1. Complete Technology Stack Deep Dive** (Rationales for Kotlin, Spring Boot 3, PostgreSQL, Flyway, Testcontainers)
3. **2. The 7 Mandatory Candidate Brief Questions & Detailed Answers** (Tied directly to code and SQL)
4. **3. End-to-End System Architecture & Data Flow**
5. **4. Complete Codebase Map ("Where is Where")** (Every package, class, and method explained)
6. **5. Concurrency Engine & State Machine Proofs** (Tuple locking, idempotency leasing, expiry reaper)
7. **6. Live Coding Modification Playbook** (Step-by-step recipes for common interviewer requests)
8. **7. Staff-Level Technical Interview FAQ** (10 deep-dive systems questions)
9. **8. Test Suite Verification & Operational Guide** (22/22 tests walkthrough & Docker Compose)

---

0. The Real-World Business Problem & Core Challenge

> **The Business Concept:**
> Nnomm is a flash-deal dining platform (similar to Swiggy Dineout, Zomato Gold, or Groupon). Premium restaurants publish exclusive, deeply-discounted food vouchers (e.g. "*70% off Michelin-Star 5-Course Tasting Menu — Only 10 vouchers available at 8:00 PM!*").
>
> This service is the **high-concurrency reservation & claim backend engine** built to guarantee absolute mathematical data integrity when thousands of diners compete for those 10 units at the exact same millisecond.

The 5 Critical Failure Modes Solved

Failure Mode	The Catastrophic Real-World Impact	How Nnomm Solves It
1. High-Concurrency Overselling	At 8:00:00 PM, 5,000 customers hit "Claim". Two separate application pods both read `stock = 1` at the same instant and both decrement it. 15 customers arrive at the restaurant with valid vouchers for only 10 seats, destroying restaurant partner trust.	**Tuple-Level Row Locking (`FOR UPDATE`) & DB Check Constraints:** PostgreSQL locks the tuple in shared memory. `CHECK (stock_available >= 0)` physically forbids negative stock at the engine level.
2. Duplicate Customer Hoarding	A scalper opens multiple browser tabs or automates scripts to claim all 10 vouchers, reselling them on black markets.	**Partial Unique Database Index:** `UNIQUE(customer_id, deal_id) WHERE status IN ('RESERVED', 'PAID', 'NEEDS_REVIEW')` rejects concurrent duplicate claims at the DB storage layer.
3. Network Dropped Mid-Request	A customer clicks "Claim", stock is deducted, but their mobile network drops before receiving the HTTP 201 response. Tapping "Claim" again would bill them twice and consume extra stock.	**2-Phase Idempotency Table:** Uses client `Idempotency-Key`. Replaying the key immediately returns the cached original claim with header `X-Idempotent-Replay: true` without touching inventory.
4. Inventory Abandonment (Hoarding)	Diners reserve vouchers and close the browser, locking inventory away from genuine buyers.	**Background Expiry Reaper (`ClaimExpiryWorker`):** Runs every 1 second using PostgreSQL `SELECT ... FOR UPDATE SKIP LOCKED` to safely reclaim expired reservations and restore stock.
5. Payment Gateway Race Conditions	Customer pays at second 599. The payment webhook arrives simultaneously with the 600-second expiry sweeper.	**Atomic Transactional State Machine:** Both paths acquire `SELECT ... FOR UPDATE` on the claim row. If payment arrives late, status transitions safely to `REFUND_PENDING`, preventing phantom vouchers.

1. Complete Technology Stack Deep Dive

Layer	Technology	Architectural Rationale & Key Benefits
Language	**Kotlin 1.9.22 (JVM 17)**	Strict compile-time null safety (prevents production NPEs on untrusted webhooks), immutable data classes, lightweight coroutines for high-concurrency simulation, and 100% interoperability with Java and Spring.
Core Framework	**Spring Boot 3.2.3**	Industry standard for enterprise microservices: Spring Web MVC for clean REST controllers, declarative validation, Spring JDBC transaction infrastructure, and `@Scheduled` background tasks.
Database	**PostgreSQL 15**	ACID transactional engine providing tuple-level pessimistic locks (`FOR UPDATE`), non-blocking worker batching (`SKIP LOCKED`), partial unique indexes, and hard database check constraints.
Connection Pool	**HikariCP**	Ultra-low-latency JVM connection pool configured via Spring Boot. Fine-tuned pool bounds (20-30 max connections, 5s timeout) prevent database resource exhaustion during flash traffic bursts.
Data Access	**Raw Parameterized SQL (JDBC)**	**Deliberately excludes Hibernate/JPA:** High-concurrency flash sales cannot use optimistic locking (which causes 99% rollback retry storms) or ORM dirty-checking overhead. Raw SQL guarantees deterministic execution plans and zero cache pollution.
Database Migrations	**Flyway 10.8.1**	Deterministic, version-controlled schema evolution. Migrations run automatically on application startup, keeping local dev, CI/CD, and Docker Compose completely synchronized.
Security & Auth	**Java-JWT & HMAC-SHA256**	`com.auth0:java-jwt` for strict HS256 JWT verification; Java `javax.crypto.Mac` with `MessageDigest.isEqual` for constant-time webhook signature verification and 300s timestamp window protection.
Testing	**JUnit 5 + Testcontainers + Embedded PostgreSQL**	Dual-engine test harness: Testcontainers PostgreSQL 15 for Dockerized CI/CD, and real Embedded PostgreSQL for zero-latency local development without Docker dependencies.
Containerization	**Docker & Docker Compose**	Multi-stage Dockerfile packaging the Spring Boot executable jar (`bootJar`) into a minimal Alpine JRE under an unprivileged user, and Docker Compose orchestrating PostgreSQL and 2 load-balanced service instances.

2. The 7 Mandatory Candidate Brief Questions & Detailed Answers

Question 1: Two customers on different instances claim the last unit at the same instant. Walk through the SQL each request runs and what PostgreSQL does with it. Why wouldn't a Kotlin Mutex or synchronized block fix this?

SQL Execution Walkthrough:
Both requests execute within `ClaimService.executeClaimTransaction` (`ClaimService.kt:L166-L219`):
1. **Transaction Start:** Both instances begin a transaction (`conn.autoCommit = false`).
2. **Row Locking:** Both run `SELECT id, restaurant_id, title, price_paise, stock_total, stock_available, starts_at, ends_at FROM deals WHERE id = ? FOR UPDATE;` (`DealRepository.kt:L42-L50`).
3. **PostgreSQL Tuple Lock:** PostgreSQL grants an exclusive row-level tuple lock (`ExclusiveLock`) to the first transaction (Tx1). Tx2 is suspended in PostgreSQL's lock queue waiting for Tx1 to complete.
4. **Tx1 Mutates & Commits:** Tx1 observes `stock_available = 1` , executes `UPDATE deals SET stock_available = stock_available - 1 WHERE id = ? AND stock_available >= 1;` , inserts the claim into `claims` , and commits. Its tuple lock is released.
5. **Tx2 Resumes & Evaluates:** PostgreSQL re-evaluates the row for Tx2 under Read Committed MVCC. Tx2 reads the freshly committed version: `stock_available = 0` . The check `deal.stock_available < quantity` triggers a `SoldOutException` (`ClaimService.kt:L186`). Tx2 rolls back, the decrement is never attempted, and 409 `SOLD_OUT` is returned.
6. **Storage Safety:** Database constraint `CHECK (stock_available >= 0)` in `V1__init_schema.sql:L9` physically prevents negative inventory.

Why Kotlin Mutex / synchronized Fails:
In-memory locks (Kotlin `Mutex` , Java `synchronized` , `ReentrantLock`) live exclusively inside a single JVM process heap. Two container instances running on different servers or Kubernetes pods have isolated memory address spaces; neither process knows about the other's mutex. Only PostgreSQL's shared lock table coordinates state across independent instances.

Question 2: Your service commits a claim, then crashes before responding. The client retries with the same Idempotency-Key. What happens? What if it crashed just before the commit?

Scenario A (Crashed After Commit):
1. The transaction committed the claim in `claims` and stored the HTTP 201 response payload in `idempotency_keys` (`status_code = 201` , `response_body = <claim JSON>`) (`ClaimService.kt:L118-L126`).
2. When the client retries with the same `Idempotency-Key` , `idempotencyRepository.findForUpdate` finds the existing record with `status_code != null` (`ClaimService.kt:L85-L88`).
3. The service immediately **replays** the cached response with header `X-Idempotent-Replay: true` . No stock is touched, no duplicate claim is created, and the customer receives their original claim.

Scenario B (Crashed Just Before Commit):
1. The database connection dropped, so PostgreSQL automatically rolled back the uncommitted transaction. No row was created in `claims` , and `stock_available` remains untouched.
2. However, the initial in-progress record in `idempotency_keys` exists with `locked_until = NOW() + 60s` (`ClaimService.kt:L74`).
3. *If retry arrives within 60s:* Returns 409 `REQUEST_IN_PROGRESS` (`ClaimService.kt:L90-L92`).
4. *If retry arrives after 60s:* The crash recovery lease has expired. `lockedUntil.isAfter(now)` evaluates to false. The service calls `idempotencyRepository.renewLock(...)` (`IdempotencyRepository.kt:L73-L89`) and safely executes the claim afresh.

Question 3: A payment.succeeded webhook and your expiry job reach the same claim in the same millisecond. List every possible ordering, with the final claim status and stock for each.

Both paths lock the claim row via `SELECT ... FOR UPDATE` (`PaymentWebhookService.kt:L47` and `ClaimRepository.kt:L80-L105`). There are 3 distinct orderings:

Execution Ordering	Internal State Transitions	Final Status	Stock Impact
Ordering 1: Webhook locks row before expiry	Webhook acquires lock while `now < expires_at` and `status == RESERVED` . Updates status to `PAID` (`PaymentWebhookService.kt:L81`). Expiry job runs later; its query `WHERE status = 'RESERVED'` skips this row.	`PAID`	**0 change** (Units held by customer)
Ordering 2: Expiry reaper locks row before webhook	Reaper locks claim, sets status to `EXPIRED` , increments deal stock by quantity (`ClaimExpiryWorker.kt:L51-L59`), and commits. Webhook acquires lock next, reads `status == EXPIRED` , and transitions status to `REFUND_PENDING` (`PaymentWebhookService.kt:L121-L127`).	`REFUND_PENDING`	**+quantity** (Restored to deal stock)
Ordering 3: Webhook locks row, but arrives after expires_at	Webhook locks row while still `RESERVED` , but detects `now >= expires_at` . It transitions status to `REFUND_PENDING` and restores stock (`PaymentWebhookService.kt:L105-L111`). Reaper skips later because claim is no longer `RESERVED` .	`REFUND_PENDING`	**+quantity** (Restored to deal stock)

Invariant Preserved: In all orderings, `stock_available + active_claims == stock_total` holds perfectly.

Question 4: Why must the webhook signature cover the raw body rather than the parsed JSON? What does the timestamp protect against, and why compare signatures in constant time?

(a) Raw Body vs. Parsed JSON:
JSON serialization is non-deterministic: key ordering (`{"a":1,"b":2}` vs `{"b":2,"a":1}`), spacing, number representation (`25000` vs `25000.0`), and unicode escaping alter byte representations. If the server deserializes into an object and re-serializes, the SHA-256 hash will not match the sender's HMAC digest. The signature must cover the exact raw byte stream as received over the wire (`CachedBodyHttpServletRequest.kt:L18`).

(b) Timestamp Replay Protection:
The `t` parameter in `X-Payment-Signature: t=1792330000,v1=...` prevents **Replay Attacks**. An attacker eavesdropping on the network could capture a valid webhook and replay it hours later. In `HmacService.kt:L27`, we enforce `abs(currentEpochSeconds - timestamp) <= 300`, rejecting any message older than 5 minutes.

(c) Constant-Time Comparison:
Standard string equality (`==` or `.equals()`) terminates early on the first mismatched byte. An attacker measuring request latency with microsecond precision can infer how many prefix bytes of a forged signature were correct, allowing byte-by-byte forgery (a *timing attack*). We use `MessageDigest.isEqual` (`HmacService.kt:L42`), which executes in constant time regardless of where the mismatch occurs.

Question 5: Which transaction isolation level do you use, and why is it enough? Name an anomaly you'd see with a weaker approach.

Level Used: `READ COMMITTED` configured in `DatabaseFactory.kt:L59`.

Why It Is Enough:
While `READ COMMITTED` is vulnerable to non-repeatable reads in ordinary queries, our mutating paths use explicit **pessimistic row locks** (`SELECT ... FOR UPDATE`) on deals (`DealRepository.kt:L43`) and claims (`ClaimRepository.kt:L48`). In PostgreSQL, `FOR UPDATE` writes an exclusive lock directly onto the row's tuple header. Concurrent transactions attempting to modify the row are forced to serialize. When a queued transaction resumes, MVCC refreshes its snapshot to read the newly committed row state. This provides strict serializability without paying the heavy abort-and-retry penalty of `SERIALIZABLE` isolation.

Anomaly Under a Weaker Approach (e.g. Plain Reads without FOR UPDATE):
Two transactions simultaneously run `SELECT stock_available FROM deals WHERE id = ?`. Both read `1`. Both compute `1 - 1 = 0`. Both execute `UPDATE deals SET stock_available = 0`. Result: 2 claims created, but stock was only decremented by 1. This is a **Lost Update / Overselling Anomaly**.

Question 6: Which constraint or index does the most work in your design? What breaks if someone drops it?

The Hardest-Working Constraint: The **Partial Unique Index** in `V1__init_schema.sql:L31-L33`:

```
CREATE UNIQUE INDEX uq_active_customer_claim ON claims (customer_id, deal_id)
WHERE status IN ('RESERVED', 'PAID', 'NEEDS_REVIEW');
```

What Breaks if Dropped:
1. A single customer firing 10 concurrent requests with different `Idempotency-Key`s across different servers would create 10 simultaneous claims.
2. Because `deals` is locked on `deal_id`, but customer claims are inserted in `claims`, two requests from the same customer could both see `hasActiveClaim = false` before either has committed, and both insert active claims.
3. Scalpers could hoard flash deals, violating: "*A customer can hold at most one active claim on a deal.*"
4. *Why Partial?* Once a claim transitions to `EXPIRED` or `REFUND_PENDING`, it drops out of the index filter, allowing the customer to legitimately participate in future claims for that deal.

Question 7: A deal goes viral and claim traffic grows 100×. What fails first, and what would you change?

What Fails First:

Single-Row PostgreSQL Lock Contention: In a flash sale, all 50,000 RPS target the exact same row in the `deals` table. Even with fast 2ms transactions, serializing 50,000 transactions on one row causes massive queueing in PostgreSQL's lock manager. The HikariCP connection pool (20-30 connections) exhausts in milliseconds, causing 503 timeouts and 100% database CPU from lock context switches.

The 3-Tier Architecture Solution:

1. **Tier 1 (Edge Caching):** Cloudflare / CDN edge caches static metadata and deal status with a 1s TTL, absorbing 90% of read traffic.
2. **Tier 2 (Redis Pre-Allocation Token Bucket):** Maintain inventory in Redis (`deal:{id}:stock = 50`). An atomic Redis Lua script checks stock, decrements, and verifies the 1-claim rule in a Redis Set in **0.2ms**. 49,950 losing requests are rejected in sub-millisecond memory without touching PostgreSQL.
3. **Tier 3 (Kafka Asynchronous Batch Persistence):** The 50 winning reservation tokens are published to an Apache Kafka topic partitioned by `deal_id` . A consumer pool inserts claims into PostgreSQL in batches with zero lock contention.

3. End-to-End System Architecture & Data Flow



4. Complete Codebase Map ("Where is Where")

File Path	Component Layer	Purpose & Exact Responsibilities
`src/main/kotlin/com/nnomm/Application.kt`	Application Entrypoint	Spring Boot `@SpringBootApplication` with `@EnableScheduling`. Boots embedded Tomcat, initializes HikariCP, runs Flyway migrations, and activates the scheduled sweeper.
`com/nnomm/controller/DealController.kt`	REST Controller	Exposes `POST /v1/deals` (restaurant admin auth) and `GET /v1/deals/{dealId}` (public deal view).
`com/nnomm/controller/ClaimController.kt`	REST Controller	Exposes `POST /v1/deals/{dealId}/claims` (customer role + Idempotency-Key) and `GET /v1/claims/{claimId}` (enforces customer/admin visibility boundaries).
`com/nnomm/controller/WebhookController.kt`	REST Controller	Exposes `POST /v1/webhooks/payments`. Reads cached raw body bytes for HMAC signature validation.
`com/nnomm/controller/DashboardController.kt`	Web Controller	Serves the visual consumer dashboard at `GET /`, `GET /health`, and dev testing helpers (`/dev/tokens`, `/dev/blast-claim`).
`com/nnomm/controller/GlobalExceptionHandler.kt`	Error Advice	`@RestControllerAdvice` that intercepts `AppException` and outputs RFC-compliant JSON errors with precise HTTP status codes (400, 401, 403, 404, 409, 413, 422).
`com/nnomm/security/PayloadSizeAndCachingFilter.kt`	Security Filter	`OncePerRequestFilter` enforcing the strict 16KB payload ceiling. Wraps the request in a cached input stream for HMAC signature checks.
`com/nnomm/security/JwtService.kt, HmacService.kt`	Security Utilities	Cryptographic services for minting/verifying HS256 tokens and constant-time HMAC-SHA256 signature calculation.
`com/nnomm/service/ClaimService.kt`	Core Concurrency Engine	Orchestrates the claim transaction: manages idempotency state machine, acquires row lock on deal, validates time window and stock, updates inventory, and saves response.
`com/nnomm/service/PaymentWebhookService.kt`	Webhook Orchestration	Handles payment event deduplication, validates signatures and timestamps, locks claim row, and transitions status (`PAID`, `NEEDS_REVIEW`, `REFUND_PENDING`).
`com/nnomm/service/ClaimExpiryWorker.kt`	Scheduled Worker	Uses `@Scheduled(fixedDelay = 1000)` and `FOR UPDATE SKIP LOCKED` to safely reclaim expired reservations and restore stock.
`com/nnomm/repository/DealRepository.kt, ClaimRepository.kt`	Data Access	Pure JDBC parameterized SQL statements for deals and claims, managing row locks and stock updates.

5. Live Coding Modification Playbook

Recipe 1: Adding a New Payment Status (e.g. Chargeback / Dispute)

1. In `domain/Models.kt`: Add `CHARGEBACK` to the `ClaimStatus` enum.
2. In `resources/db/migration/V2__add_chargeback_status.sql`: Update the check constraint:

```
ALTER TABLE claims DROP CONSTRAINT claims_status_check;
ALTER TABLE claims ADD CONSTRAINT claims_status_check
  CHECK (status IN ('RESERVED', 'PAID', 'EXPIRED', 'NEEDS_REVIEW', 'REFUND_PENDING', 'CHARGEBACK'));
```

3. In `service/PaymentWebhookService.kt`: Add the event branch for `payment.disputed` or `payment.chargeback`.

Recipe 2: Changing the Reservation Hold Window (TTL)

The claim hold window is completely configurable without modifying code:

- Set the environment variable `CLAIM_TTL_SECONDS=300` in your environment or `docker-compose.yml`.
- Or update `nnomm.claim-ttl-seconds: 300` in `application.yml`.

Recipe 3: Adding a New Field to Deal (e.g. `description: String`)

1. In `src/main/resources/db/migration/V2__add_description_to_deals.sql`:

```
ALTER TABLE deals ADD COLUMN description TEXT DEFAULT '';
```

2. In `domain/Models.kt`: Add `val description: String = ""` to `Deal` and `CreateDealRequest`.
3. In `repository/DealRepository.kt`: Add `description` to the `INSERT INTO deals (...)` SQL and row mapper.

6. Automated Test Suite Walkthrough (22/22 Passing)

The test suite validates every concurrency, security, and lifecycle invariant against real PostgreSQL:

```
$ ./gradlew test --no-daemon

AuthAndVisibilityTest > testRoleRestrictionsOnEndpoints() PASSED
AuthAndVisibilityTest > testClaimVisibilityRestrictions() PASSED
AuthAndVisibilityTest > testJwtTokenSecurity() PASSED
ConcurrencyTest > test200ConcurrentClaimsNeverOversell() PASSED
ConcurrencyTest > testPaymentAndExpiryRaceLeavesStockInvariantIntact() PASSED
ConcurrencyTest > testConcurrentRequestsWithSameIdempotencyKey() PASSED
ConcurrencyTest > testConcurrentClaimsBySameCustomerWithDifferentKeys() PASSED
LifecycleAndExpiryTest > testDealNotLiveBeforeOrAfter() PASSED
LifecycleAndExpiryTest > testClaimExpiryReclaimsUnitsAndHidesReservedStatus() PASSED
LifecycleAndExpiryTest > testReplayedConflictStaysConflict() PASSED
LifecycleAndExpiryTest > testIdempotencyReusedWithDifferentParameters() PASSED
SpringMvcWebTest > testHealthEndpoint() PASSED
SpringMvcWebTest > testDashboardEndpoint() PASSED
SpringMvcWebTest > testDealCreationAndClaimingWorkflow() PASSED
SpringMvcWebTest > testPayloadTooLargeRejected() PASSED
SpringMvcWebTest > testDevTokensEndpoint() PASSED
SpringMvcWebTest > testPaymentWebhookController() PASSED
WebhookTest > testBadSignatureRejected() PASSED
WebhookTest > testPaymentAmountMismatchMovesToNeedsReview() PASSED
WebhookTest > testPaymentFailedDoesNotChangeStatus() PASSED
WebhookTest > testStaleTimestampRejected() PASSED
WebhookTest > testDuplicateEventsAreIdempotent() PASSED

BUILD SUCCESSFUL: 22 tests executed, 22 passed (100% pass rate)
```

Running Locally & Docker Compose

```
# 1. Standalone Dev Mode with Embedded PostgreSQL (Zero setup):
./gradlew devRun
# On Windows: gradlew.bat devRun
# Open: http://localhost:8080/

# 2. Multi-Instance Docker Compose (Postgres + 2 Spring Boot instances):
docker compose up --build
curl http://localhost:8080/health
curl http://localhost:8081/health
```